TICKETSTORE

Online Ticket Booking System

Technical Documentation & Project Report

Object-Oriented Programming I

Department of Computer Science & Engineering

Submitted by:
Azmain Mohib Khan (2025-2-60-089)
Md. Ashikur Rahman (2025-2-60-070)
Md. Emon (2025-2-60-093)
Md. Ridowan Ul Haque (2025-2-60-111)

1. Introduction

TicketStore is a Java-based desktop application that we built as our first-year OOP course project. The idea came from wanting to make something practical — a system that people could actually use to browse events and book tickets without any unnecessary complication.

The application handles everything from user registration and login to browsing events, booking tickets, and viewing your booking history. There is also a separate admin side where privileged users can manage events, view all bookings system-wide, and generate a sales report. The whole thing runs as a Swing GUI, which we chose because it is built into Java and did not require installing anything extra.

1.1 Objectives

The main goals we set out to achieve with this project were:

- Build a fully functional ticket booking system that normal users can navigate without needing any instructions.
- Implement role-based access so regular users and admins have completely different experiences within the same application.
- Persist all data across sessions using flat-file storage, since we had not yet covered databases.
- Apply the OOP concepts we learned in class — encapsulation, separation of concerns, and clean class design.

1.2 Scope

The system covers the following features:

- User registration with input validation and login with session management.
- Browsing a list of events with search and category filter functionality.
- Booking tickets with a quantity selector and a simulated payment confirmation step.
- Viewing and cancelling personal bookings, with automatic seat restoration on cancellation.
- Admin panel with full CRUD operations on events.
- Admin access to all bookings and a generated sales report broken down by event.

1.3 Technologies Used

Component	Technology / Version
Programming Language	Java (JDK 11+)

Component	Technology / Version
GUI Framework	Java Swing (javax.swing)
Look & Feel	Metal Look and Feel (javax.swing.plaf.metal)
Data Storage	Plain-text CSV flat files
Build Tool	Standard javac / IDE (IntelliJ / Eclipse)
Target Platform	Desktop (Windows / macOS / Linux)

2. System Architecture

The project is organized into three main packages — model, data, and gui — plus a top-level Main class that boots the application. This separation was intentional; we wanted to keep the data logic completely independent of the UI so that either side could be changed without breaking the other.

2.1 Package Structure

```
TicketStore/
  Main.java                    -- Entry point
  model/
    User.java                  -- User data model
    Event.java                 -- Event data model
    Booking.java               -- Booking data model
  data/
    DataManager.java           -- File I/O and all CRUD operations
    Session.java               -- Static session / auth state
  gui/
    MainFrame.java             -- Root JFrame, CardLayout navigation
    Theme.java                 -- Centralized design system
    NavBar.java                -- Reusable top navigation bar
    LoginPanel.java            -- Login screen
    RegisterPanel.java         -- Registration screen
    EventsPanel.java           -- Event browsing and booking
    BookingsPanel.java         -- User booking history
    AdminPanel.java            -- Admin management interface
  data_files/
    users.txt                  -- Persistent user records (CSV)
    events.txt                 -- Persistent event records (CSV)
    bookings.txt               -- Persistent booking records (CSV)
```

2.2 Architectural Pattern

We followed a layered architecture loosely based on the MVC (Model-View-Controller) pattern, adapted for a Swing desktop application:

Layer	Classes	Responsibility
Model	User, Event, Booking	Data structure, CSV serialization, domain logic (bookSeat, cancel)
Data Layer	DataManager, Session	File I/O, CRUD operations, seeding, session state
View / Controller	All gui.* classes	UI rendering, user interaction, calling DataManager

The model classes know nothing about the GUI or files — they just hold data and provide a few domain-level methods. DataManager sits in the middle and handles reading and writing everything to disk. The GUI panels call DataManager directly for data and use Session to check who is logged in and what they are allowed to do.

2.3 Navigation Flow

MainFrame uses a CardLayout to switch between screens. When showCard() is called, it first triggers a refresh() on the target panel (so data is always fresh before display), then flips the card. The five screens and their flow are:

Screen	Accessible By	Navigates To
Login	Everyone (unauthenticated)	Register, Events (on success)
Register	Everyone (unauthenticated)	Login
Events	All logged-in users	Bookings, Admin (if admin)
Bookings	All logged-in users	Events, Admin (if admin)
Admin Panel	Admin users only	Events, Bookings

3. Class Descriptions

3.1 Model Layer

3.1.1 User.java

Represents a registered user of the system. Holds the username, password (plain text), email, full name, and an isAdmin boolean flag. Provides toCSV() and a static fromCSV() for file persistence. The class is intentionally simple — it is a data container with no business logic.

Field	Type	Description
username	String	Unique login identifier
password	String	User password (stored as plain text)
email	String	User email address
fullName	String	Display name shown in the NavBar
isAdmin	boolean	Grants access to the Admin Panel if true

3.1.2 Event.java

Represents a bookable event. Tracks both totalSeats and availableSeats separately so we can always calculate how many tickets have been sold. The bookSeat() method decrements available seats and returns false if the event is sold out, and cancelSeat() increments it back up on cancellation (capped at totalSeats). All fields are mutable through setters so the admin can edit events without creating new objects.

Field	Type	Description
eventId	String	Unique identifier, e.g. EV001
name	String	Event title
type	String	Category: Movie, Concert, Theater, or Sports
location	String	Venue name
date / time	String	Formatted as yyyy-MM-dd and HH:MM
price	double	Ticket price per seat
totalSeats	int	Original total capacity
availableSeats	int	Remaining seats (decrements on booking)
description	String	Short event description

3.1.3 Booking.java

Represents a ticket booking made by a user. The static create() factory method is the preferred way to instantiate a Booking — it auto-generates a unique booking ID using System.currentTimeMillis() and stamps the current date/time. Status is either CONFIRMED or CANCELLED, and the cancel() method flips it. The CSV format uses a limit of 8 parts in the split call to safely handle event names that might contain commas.

Field	Type	Description
bookingId	String	Auto-generated: BK + timestamp
username	String	Owner of the booking
eventId	String	Reference to the booked event
eventName	String	Denormalized name for display without lookups
numTickets	int	Number of tickets in this booking
totalAmount	double	Price × numTickets at time of booking
bookingDate	String	Timestamp formatted as yyyy-MM-dd HH:mm
status	String	CONFIRMED or CANCELLED

3.2 Data Layer

3.2.1 DataManager.java

This is the backbone of the application's persistence. It manages three flat-file CSV stores: users.txt, events.txt, and bookings.txt, all located inside a data_files directory that is created on first run. The initialize() method is called once at startup; it creates the directory and files if they do not exist, seeds a default admin account if no admin is found, and seeds eight sample events if the events file is empty.

All read operations load the entire file each time (since the dataset is small), deserialize each line using the model's fromCSV(), filter out any null results from malformed lines, and return a List. Write operations serialize the entire list back to the file, overwriting it completely. This is not efficient for large datasets but is simple and reliable for a course project.

Method Group	Key Methods
Initialization	initialize(), ensureFile(), seedAdminIfNeeded(), seedEventsIfNeeded()
User Operations	loadUsers(), saveUsers(), findUser(), usernameExists(), addUser()
Event Operations	loadEvents(), saveEvents(), findEvent(), updateEvent(), addEvent(), deleteEvent(), generateEventId()
Booking Operations	loadBookings(), saveBookings(), addBooking(), getUserBookings(), updateBooking()

3.2.2 Session.java

A simple static class that holds the currently authenticated User object in memory. It has four methods: login(user), logout(), getCurrentUser(), and two convenience checkers — isLoggedIn() and isAdmin(). Since it is entirely static, any part of the application can query the session state without passing a reference around. Session state is only reset when logout() is called or the application is closed.

3.3 GUI Layer

3.3.1 MainFrame.java

The root application window. It extends JFrame and sets up a CardLayout over a single JPanel to hold all five screens. Each screen is instantiated once at startup and held as a field. The showCard() method is the central navigation hub — it refreshes the target panel before showing it, ensuring data displayed is always up to date. The logout() method clears the session, resets the LoginPanel fields, and navigates back to the login screen.

3.3.2 Theme.java

A utility class that defines the visual identity of the entire application. It stores all colors as static Color constants (a deep navy palette with electric amber as the accent) and all fonts as static Font constants. It also provides four factory methods — makeButton(), makeField(), makePasswordField(), and makeLabel() — so every panel uses identical-looking components without duplicating styling code. The makeButton() method supports a primary/secondary distinction and includes hover effects.

Constant	Value	Usage
BG_DARK	RGB(12, 18, 40)	Main window and panel backgrounds
BG_PANEL	RGB(20, 30, 60)	Card and dialog backgrounds
BG_CARD	RGB(28, 42, 80)	Table backgrounds
ACCENT	RGB(255, 176, 0)	Buttons, highlights, logo color
SUCCESS	RGB(60, 210, 130)	Confirmed booking status text
DANGER	RGB(255, 80, 80)	Cancelled status, delete actions
TEXT_PRIMARY	RGB(240, 240, 250)	Main body text
TEXT_MUTED	RGB(140, 150, 180)	Labels, secondary info

3.3.3 NavBar.java

A reusable JPanel rendered at the top of every post-login screen. It shows the TicketStore logo on the left, navigation links in the center, and the current user's full name with a Logout button on the right. The navigation links shown depend on the user's role — the Admin Panel link only appears if Session.isAdmin() returns true. The currently active screen's link is rendered in the accent color and bold; the others are muted and gain a hover highlight.

3.3.4 LoginPanel.java

The entry point of the application for users. It presents a centered card with the app name, a username field, a password field, an inline error label, a Sign In button, and a Create Account button. The doLogin() method validates that both fields are filled, looks up the user via DataManager.findUser(), and on success calls Session.login() and navigates to the Events screen. The Enter key on the password field also triggers login.

3.3.5 RegisterPanel.java

Handles new user registration. Collects full name, username, email, password, and confirm password. Validates that no field is empty, passwords match, the password is at least 6 characters, and the username is not already taken. On success it calls

DataManager.addUser() and redirects to the Login screen. Errors are shown inline below the form fields.

3.3.6 EventsPanel.java

The primary screen for regular users. It loads all events from DataManager into a table on each refresh. The search bar filters by event name or location (case-insensitive), and the dropdown filters by event type. The available seats column is colour-coded: green for plentiful, amber for fewer than 20, and red for sold out. Clicking Book Selected Event opens a dialog showing event details and a ticket quantity spinner (capped at 10 or available seats, whichever is lower), followed by a payment confirmation. On confirmation, the seats are decremented, the event is updated, and a Booking record is created and saved.

3.3.7 BookingsPanel.java

Shows the logged-in user's booking history filtered by their username. Displays a summary at the bottom showing the number of confirmed bookings and total amount spent. The Cancel Booking button checks that the selected booking is not already cancelled, asks for confirmation, flips the booking status to CANCELLED, calls event.cancelSeat() to restore the seat, and saves both records back to disk.

3.3.8 AdminPanel.java

The admin interface, only accessible to users with isAdmin set to true. It contains three tabs:

- – A table of all events with Add, Edit, and Delete buttons. The Add/Edit dialog uses a form with a dropdown for event type and text fields for all other attributes. Input is validated before saving.
- – A read-only table of every booking in the system, with a footer showing total booking count and total confirmed revenue.
- – A plain-text report rendered in a monospaced font showing overall stats and a per-event breakdown of tickets sold, seats remaining, and revenue generated. A Refresh button regenerates it on demand.

4. Data Storage

4.1 File Format

All data is stored in plain-text CSV files inside the data_files directory. Each line represents one record. The model classes handle their own serialization through toCSV() and fromCSV() methods, keeping the storage logic close to the data it describes.

4.1.1 users.txt

```
Format: username,password,email,fullName,isAdmin
Example: admin,admin123,admin@tickets.com,Administrator,true
         john,pass1234,john@email.com,John Smith,false
```

4.1.2 events.txt

```
Format:
eventId,name,type,location,date,time,price,totalSeats,availableSeats,descrip
tion
Example: EV001,Avengers Finale,Movie,Cineplex
Downtown,2026-05-10,18:00,12.5,200,198,Epic Marvel conclusion
```

4.1.3 bookings.txt

```
Format:
bookingId,username,eventId,eventName,numTickets,totalAmount,bookingDate,stat
us
Example: BK1715000000000,john,EV001,Avengers Finale,2,25.0,2026-04-15
14:30,CONFIRMED
```

4.2 Seed Data

On first run, DataManager automatically seeds one admin account and eight events so the application is immediately usable without any manual setup:

Event ID	Event Name	Type	Price	Seats
EV001	Avengers Finale	Movie	$12.50	200
EV002	Ed Sheeran Live	Concert	$85.00	5000
EV003	Hamilton Musical	Theater	$65.00	400
EV004	NBA Finals Game 1	Sports	$120.00	1000
EV005	Taylor Swift Eras	Concert	$150.00	8000
EV006	Dune: Part Three	Movie	$15.00	300

Event ID	Event Name	Type	Price	Seats
EV007	World Chess Championship	Sports	$30.00	500
EV008	Comedy Night Gala	Theater	$40.00	250

5. Features & Functionality

5.1 User Features

Feature	Description
Account Registration	New users register with full name, username, email, and password. Duplicate username detection and password strength check are included.
Login / Logout	Credentials are verified against the stored records. Session state is maintained statically until logout.
Browse Events	All events are displayed in a sortable table. Users can search by name or location and filter by event category.
Book Tickets	Users select an event, choose quantity (up to 10 per booking), confirm a payment simulation, and receive a Booking ID on success.
View Bookings	A personal booking history showing event name, number of tickets, total paid, booking date, and current status.
Cancel Booking	Users can cancel any confirmed booking. The seat count is automatically restored to the event.

5.2 Admin Features

Feature	Description
Add Event	Admin fills a form with all event details. Event ID is auto-generated in the format EV00X.
Edit Event	Admin selects an existing event and modifies any field. Available seats are not reset when editing — only totalSeats target is editable.
Delete Event	Admin deletes an event after a confirmation dialog. Associated bookings remain in the file for record-keeping.
View All Bookings	Full list of all bookings across all users, with a running total of confirmed revenue.
Sales Report	Auto-generated text report with system-wide stats and per-event breakdown. Refreshable on demand.

6. How to Run the Application

6.1 Prerequisites

- Java Development Kit (JDK) 11 or newer installed.
- Any Java IDE (IntelliJ IDEA, Eclipse, NetBeans) or command-line access.
- No external libraries or dependencies required — the project uses only the Java standard library.

6.2 Running from an IDE

1. Open the project in your IDE.
2. Set Main.java as the run configuration's main class.
3. Run the project. The data_files/ directory will be created automatically on first launch.

6.3 Running from the Command Line

```
# Compile all source files
javac -d out src/**/*.java src/Main.java

# Run the application
java -cp out Main
```

6.4 Default Admin Credentials

Field	Value
Username	admin
Password	admin123

These credentials are seeded automatically on the first run if no admin account exists in users.txt.

7. OOP Concepts Applied

Since this is a course project for Object-Oriented Programming, we want to highlight where the concepts we learned in class actually show up in the code.

7.1 Encapsulation

All three model classes — User, Event, and Booking — keep their fields private and expose them only through getters. Event additionally provides setters to allow admin edits, but only the fields that make sense to change (you cannot, for example, set an eventId after construction). The domain logic for seat management (bookSeat, cancelSeat) lives inside the Event class rather than being handled in the GUI.

7.2 Abstraction

DataManager abstracts away all file I/O details from the rest of the application. The GUI panels never open a BufferedReader or write to a file directly — they just call methods like DataManager.addBooking() or DataManager.loadEvents() without caring about how storage works underneath. Similarly, Theme abstracts all visual styling so individual panels do not contain any colour or font constants.

7.3 Separation of Concerns

The three-package structure enforces a clear separation: model classes describe what data looks like, the data package handles how data is stored and retrieved, and the gui package handles how data is presented to the user. Changes in one layer do not require changes in the others — for instance, we could switch from flat-file storage to a database by only changing DataManager, without touching any GUI code.

7.4 Reusability

NavBar and Theme are the best examples of reusability in this project. NavBar is instantiated fresh in every panel's buildUI() method, always showing the correct active tab for its context. Theme's factory methods (makeButton, makeField, etc.) are called dozens of times across all panels, ensuring the UI looks consistent everywhere with zero duplication of styling code.

7.5 Static Factory Methods

Both Booking.create() and DataManager.generateEventId() are static factory methods — a pattern we used to centralize object construction logic that involves side effects (like reading files or checking the system clock) that do not belong in a constructor.

8. Known Limitations

We want to be upfront about what the current version does not handle well. These are things we are aware of and would fix if this were a production system.

- Passwords are stored in plain text in the CSV files. A proper system would hash passwords using something like BCrypt before storing them.
- There is no protection against concurrent access. If two instances of the application ran at the same time and both tried to write to the same file, data could be corrupted.
- The CSV parsing is fragile for fields that themselves contain commas (for example, an event name like "Music, Dance & More" would break the split). A proper CSV parser would handle quoting.
- Event IDs are generated by finding the current maximum and incrementing it. If an event is deleted, that ID is never reused, which is fine, but if two users added events simultaneously the same ID could be generated twice.
- The Sales Report tab does not refresh automatically — the admin needs to click the Refresh button to see updated figures.
- There is no email verification for the registration form; any string passes as a valid email.

9. Conclusion

TicketStore ended up being a much more complete project than we initially expected. Starting from just the idea of "a system where you can book event tickets", we ended up building something with full user authentication, role-based access control, CRUD admin functionality, a search and filter interface, and persistent data storage — all without using a single external library.

The biggest thing we took away from this project is how much the structure of your code matters. Early on, we tried putting data logic directly in the GUI and it quickly became hard to follow. Once we separated things into model, data, and gui packages it all became much easier to work on in parallel and debug. The Theme class in particular saved a huge amount of time — changing the colour scheme once was enough to update every screen at once.

If we were to extend this project further, the first things we would add are a proper database backend, password hashing, and an email confirmation on booking. We would also look at adding a seat map view for events and a PDF receipt generator for confirmed bookings.